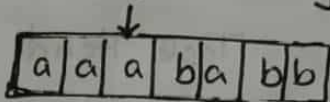
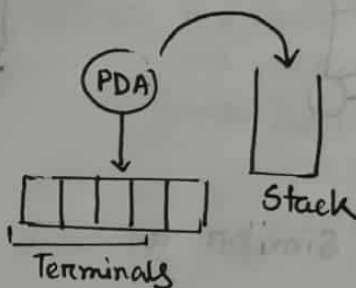


Turing Machine

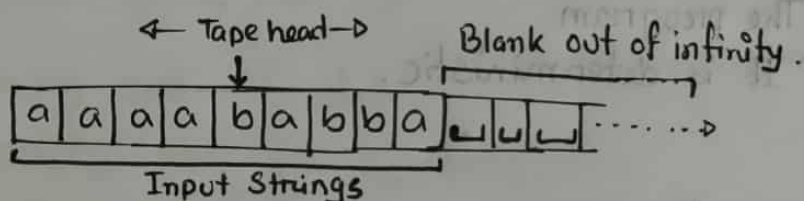
(FSM) → The input string



(PDA) → The input string
→ A Stack



Turing Machine → A Tap



Tap Alphabets: $\Sigma \{0, 1, a, b, x, z_0\}$

The blank $\sqcup$ is a special symbol. $\sqcup \notin \Sigma$

The blank is a special symbol used to fill the infinite tape.

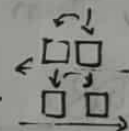
Operations on the Tap:

→ Read / scan symbol below the tape head. $\boxed{a}$ Read/scan → a

→ Update / write a symbol below the tape head. $\boxed{a} \xrightarrow{\text{update}} \boxed{b}$

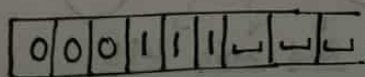
→ Move the tape head one step left.

→ Move the tape head one step Right.



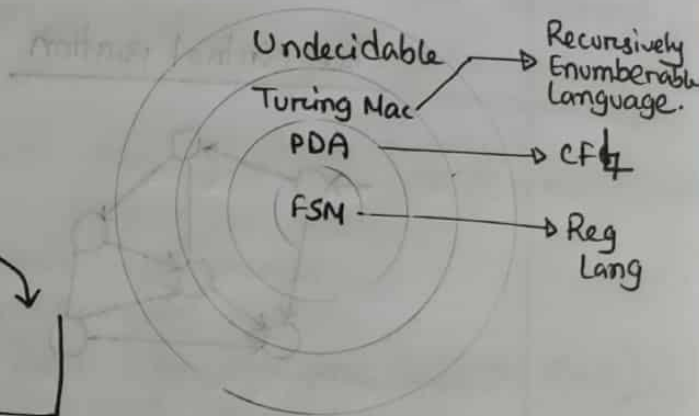
Here we put our needed string ref to the tape and then test that to solve a problem.

Ex. $0^n 1^n \mid n \geq 1$

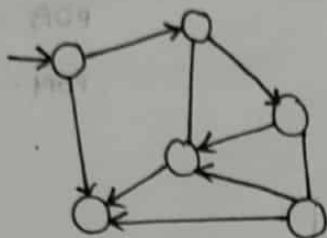


then we draw Diagram.

- Tap can be ∞ (not on both side)
- TM can Acc/Reg any time no need to read full str.
- Deterministic Machine.

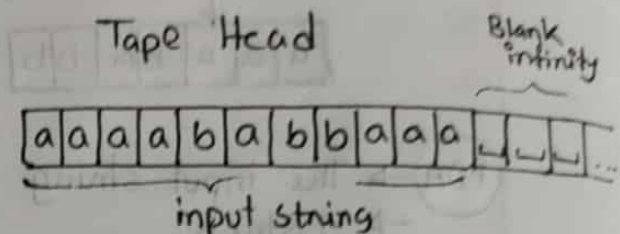


The control portion



The control portion similar to FSM or PDA.

- ↳ The program
- ↳ It is deterministic.

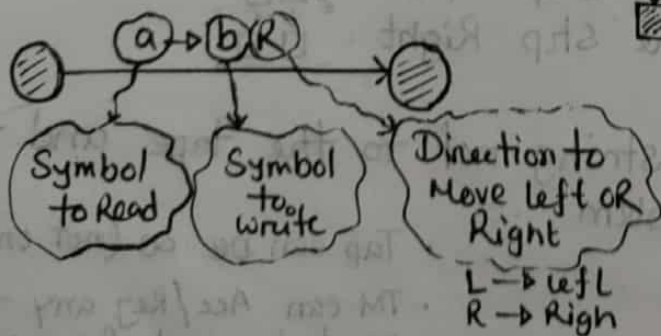


Rule of operation 1:

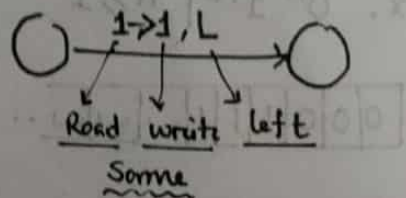
At each step of the computation,

- Read the current symbol.
- update (write) the same cell.
- Move exactly one cell either left or right.

If we are at the left end of the tape, and trying to move left, then do not move. Stay at the left end.



If don't want to update the cell, Just write the same symbol.



Rule of operation 2

→ Control is with a sort of PSM.

→ Initial state.

→ Final state → Accept State.

→ Reject State.

→ Computation can either → Halt and Accept.

→ Halt and Reject,

→ loop (the machine fail to Halt).

TM can't decide

(TM)

if Halt → Turing decidable lang / Recursive lang

if Halt(Accept) → Turing Recognizable
Recognizably Enumerable lang.

Halt → when it come to a stop.

The Turing machine can be defined as a set of 7 tuples.

$(Q, \Sigma, \Gamma, \delta, q_0, b, F)$

Q → Non empty set of states.

Σ → Non empty set of symbols.

Γ → Non empty set of tape symbols.

δ → Transition function defined as $Q \times \Sigma \rightarrow \Gamma \times (\mathbb{N} \times \mathbb{N}) \times Q$

q_0 → Initial State.

b → Blank symbol.

F → Set of final state (Accept/Reject)

Thus, the production rule of Turing Machine will be written as,

$\delta(q_0, a) \rightarrow \delta(q_1, y, R)$

Turing thesis → any computation that can be carried out by mechanical means can be performed by some Turing machine.

Few arguments for accepting this thesis are:

- 1) Anything that can be done on existing digital computer can also be done by TM.
- 2) No one has yet been able to suggest a problem Solvable by what we consider an algorithm, for which a Turing Machine Prog can not be written.

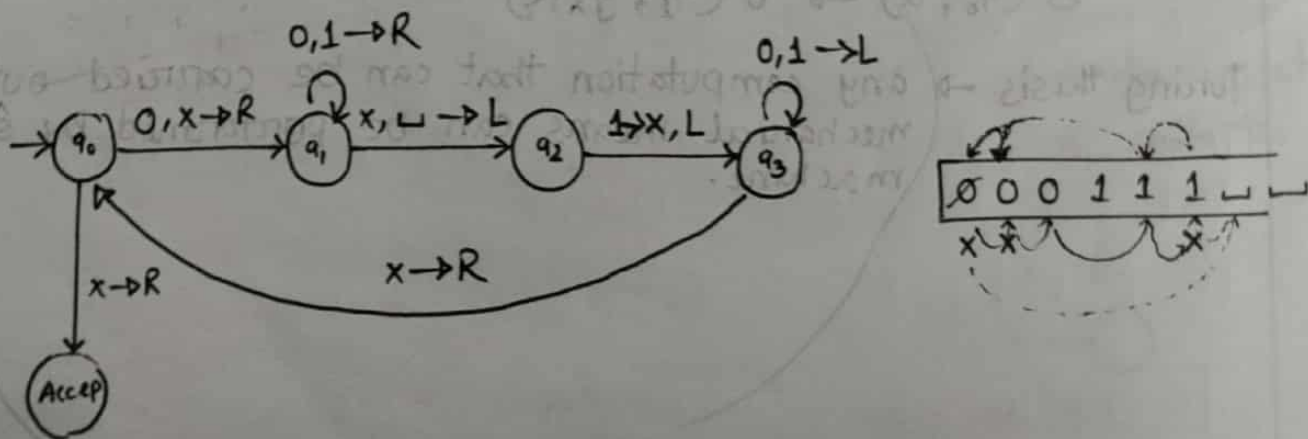
Recursively Enumerable Lang

A lang L and Σ is said to be Recursively enumerable if there exists a Turing Machine that accepts it.

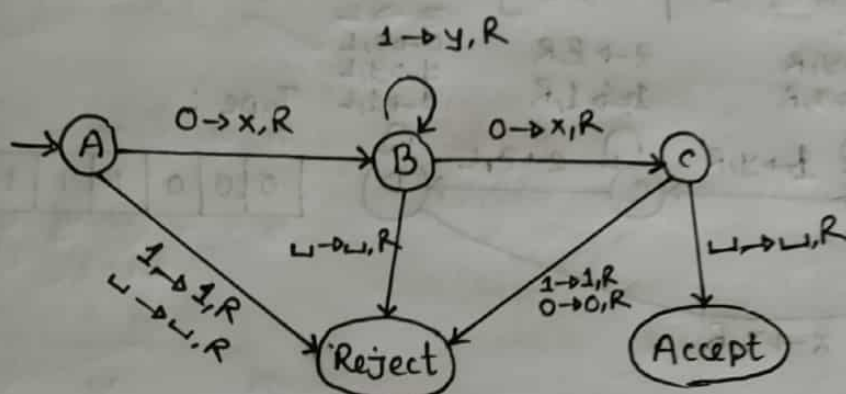
Example - 2

$$L = \{0^n 1^n \mid n > 0\}$$

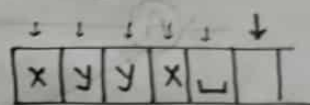
$L(M) \rightarrow \{w \in \Sigma^* \mid w \text{ contains equal num of 0's and 1's}\} \times$



Example - 1 Design a TM which recognizes the lang $L = 01^*0$



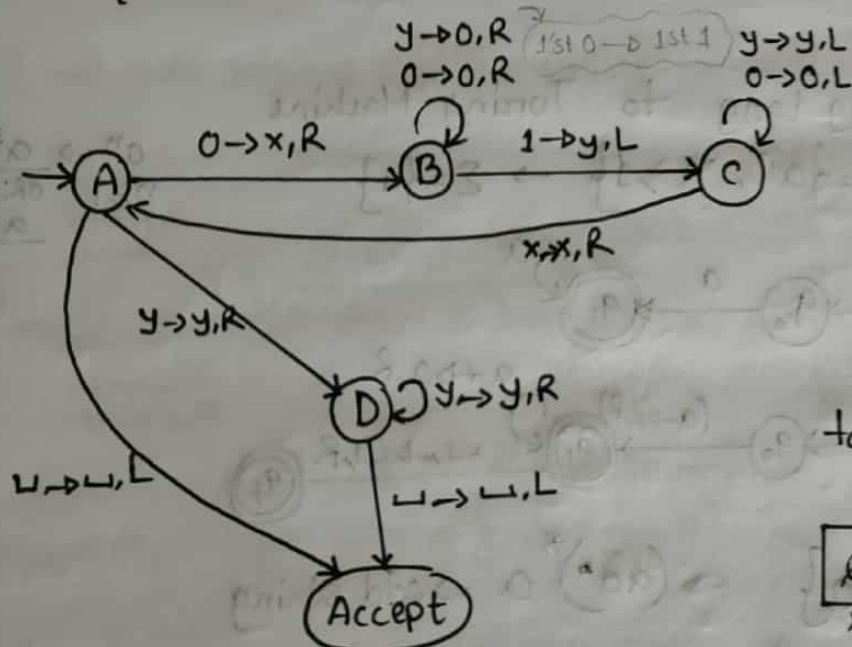
tape:



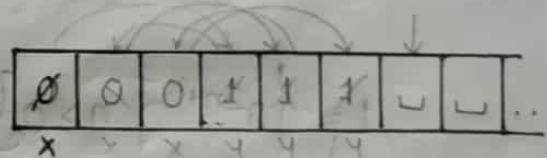
St: 0110

Example - 2 $L = \{0^n 1^n \mid n \geq 0\}$

$L(M) \rightarrow \{w \in \Sigma^* \mid w \text{ containing equal num of 0's and 1's}\}, \Sigma = \{0, 1\}$

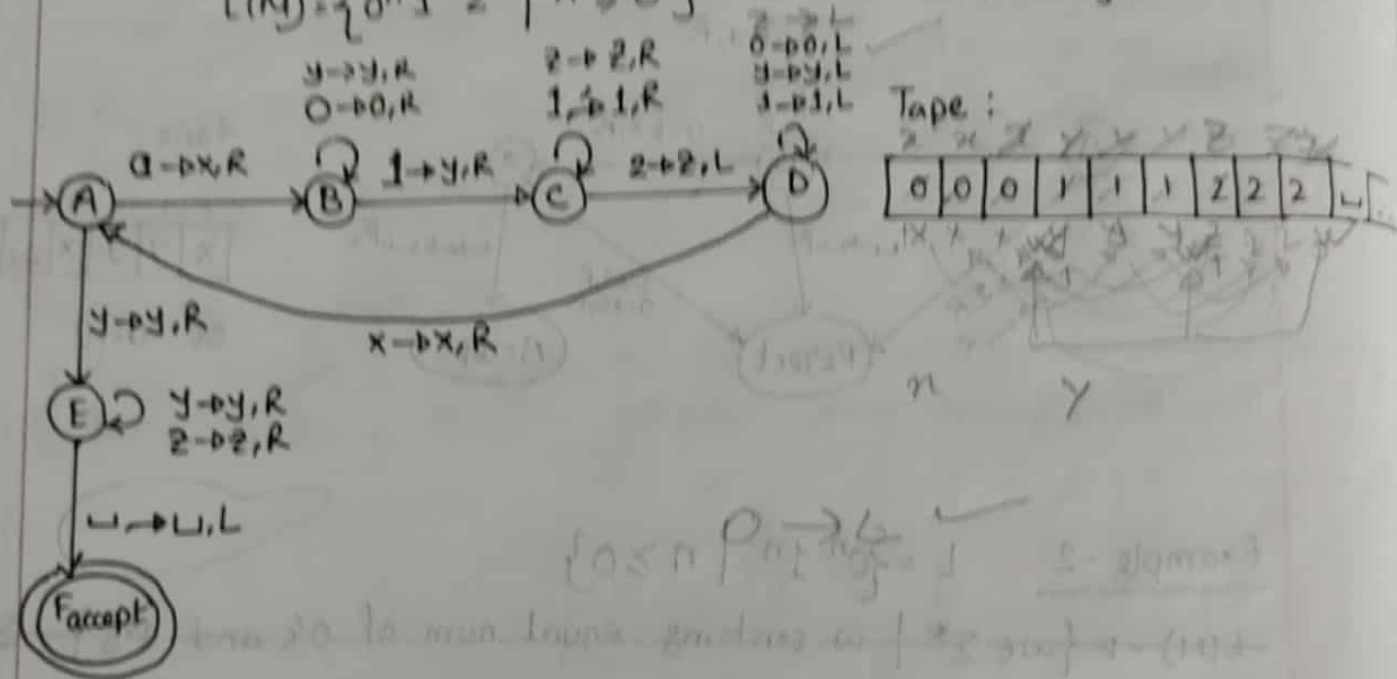


tape:



Example 3 - $\{a^n b^n c^n \mid n \geq 1\} = L$

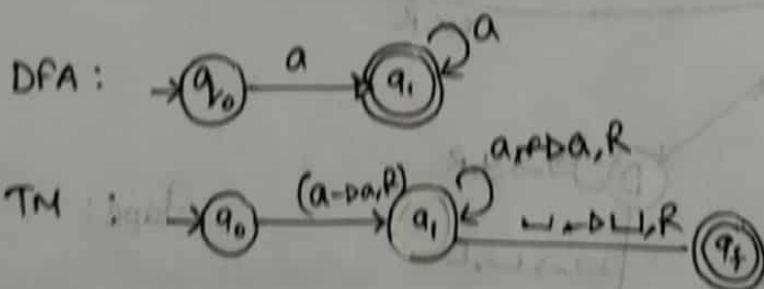
$L(N) = \{0^n 1^n 2^n \mid n \geq 0\}$ where $\Sigma = \{0, 1, 2\}$



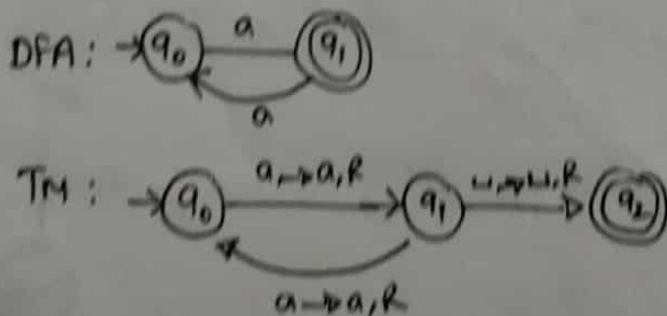
Example 4 - Reg lang to Turing Machine

$L = \{a^n, n \geq 1\} \rightarrow \Sigma = \{a\}$

$a^n \rightarrow \frac{a^* a}{\text{or } a a^*}$

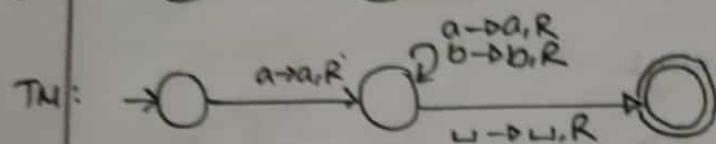
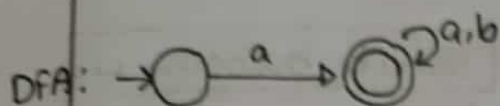


$L = \{a^{2n+1} \mid n \geq 0\} \rightarrow (aa^*)^* a \rightarrow \text{odd string}$

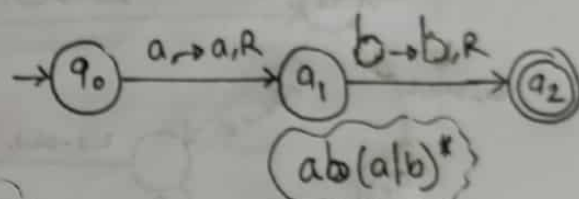
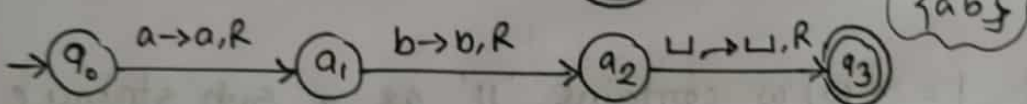
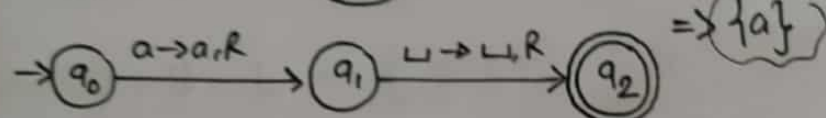
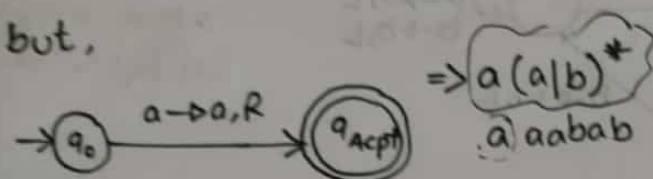


$L = \{\text{start with } a\}; \Sigma = \{a, b\}$

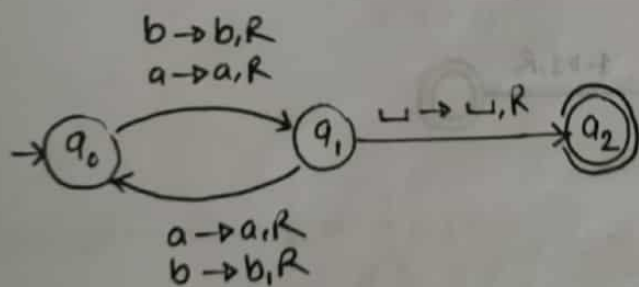
if only Right move $\rightarrow$ Regular language
L, R both $\rightarrow$ can be still Reg



but,

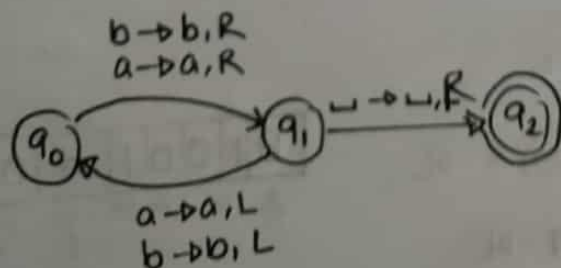


Exm-5) $L = \{\text{all odd string of } \Sigma = \{a, b\}\}$



odd all strings
 $\{a, b, aaa, aaaaa, aba, aab, abb, baa, bba, bab, bbb, \dots\}$

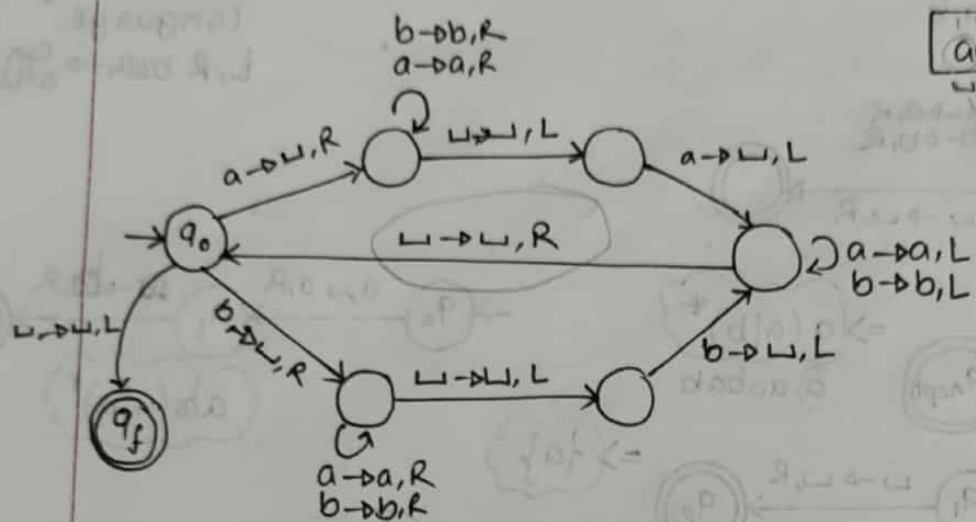
Exp-6) $L = \{\text{only one length string}\} \Sigma = \{a, b\}$



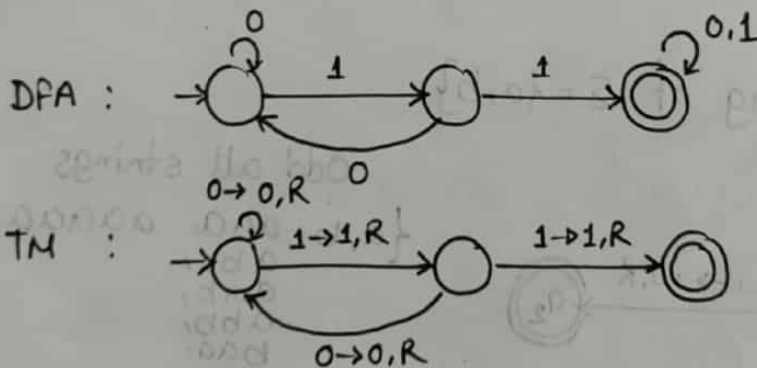
$w \# w \rightarrow$ PDA Not possible
Need TM.

$w \# w \rightarrow$ PDA Not possible
Need TM.

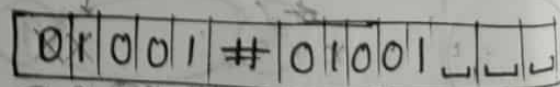
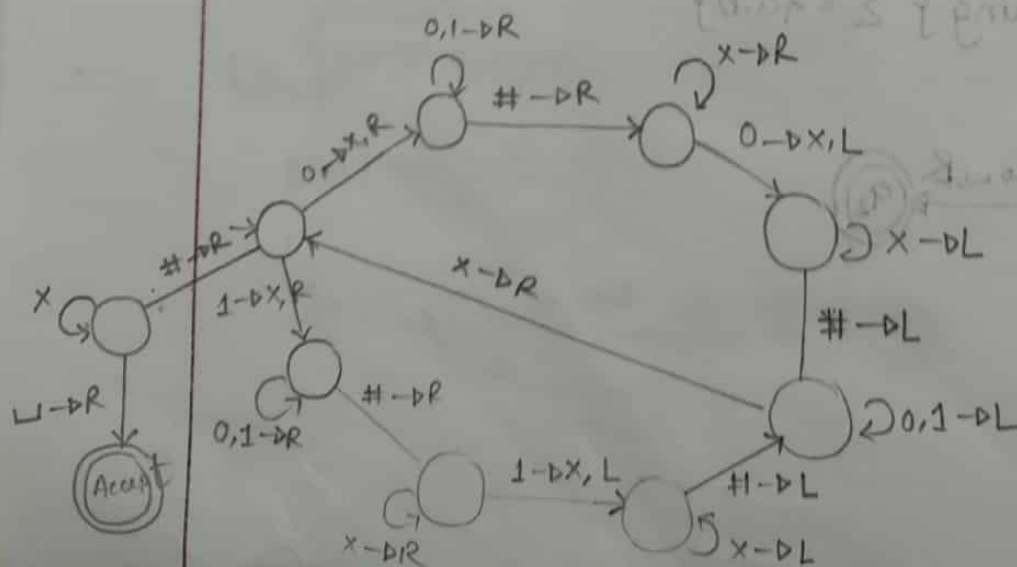
Example - 7. $L = \{ww^R\}$ $w \in \{a, b\}^*$



Example - 8 $L = \{w \mid w \text{ contains '11' as a sub string}\}$

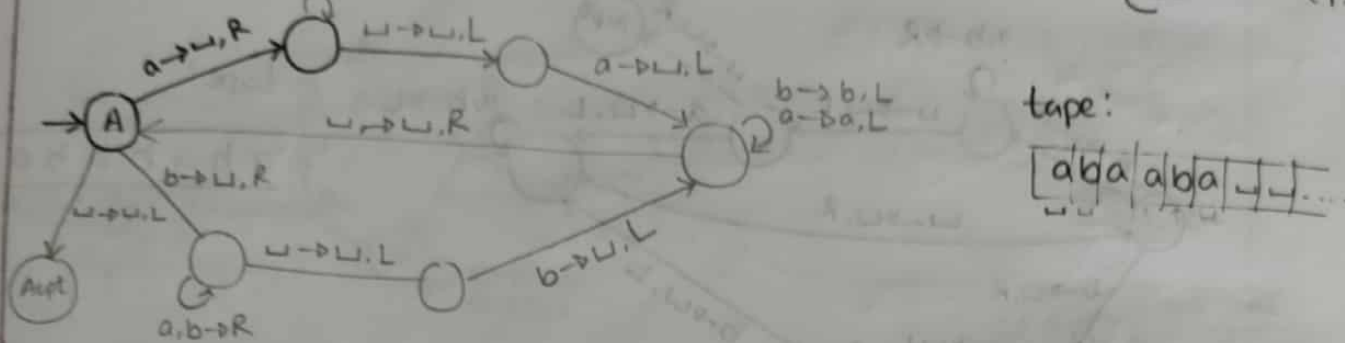


Example - 9: $L = \{\omega \# \omega\} \quad \omega \in \Sigma^*$



$\begin{cases} \text{odd palin} \rightarrow \text{racecar} \\ \text{even palin} \rightarrow \text{noon} \end{cases}$

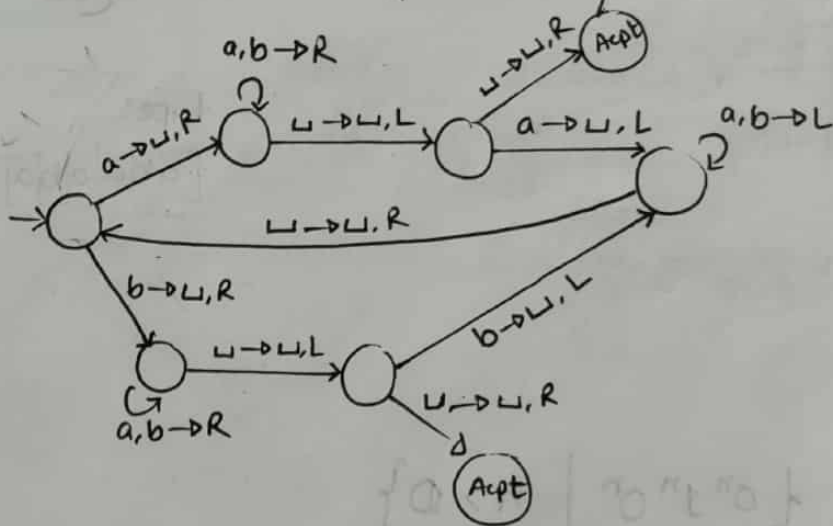
Example - 10 $L = \{ \text{Even Palindromes} \}$ $\Sigma = \{a, b\}$
 $\begin{cases} \text{even} \rightarrow ww^R \\ \text{odd} \rightarrow w(a|b)w^R \end{cases}$



Example - 11. $L = \{ 0^n 1^n 0^n \mid n \geq 0 \}$

Example - 12. $L = \{ w \# w \mid w \in \{a, b, c\}^* \}$

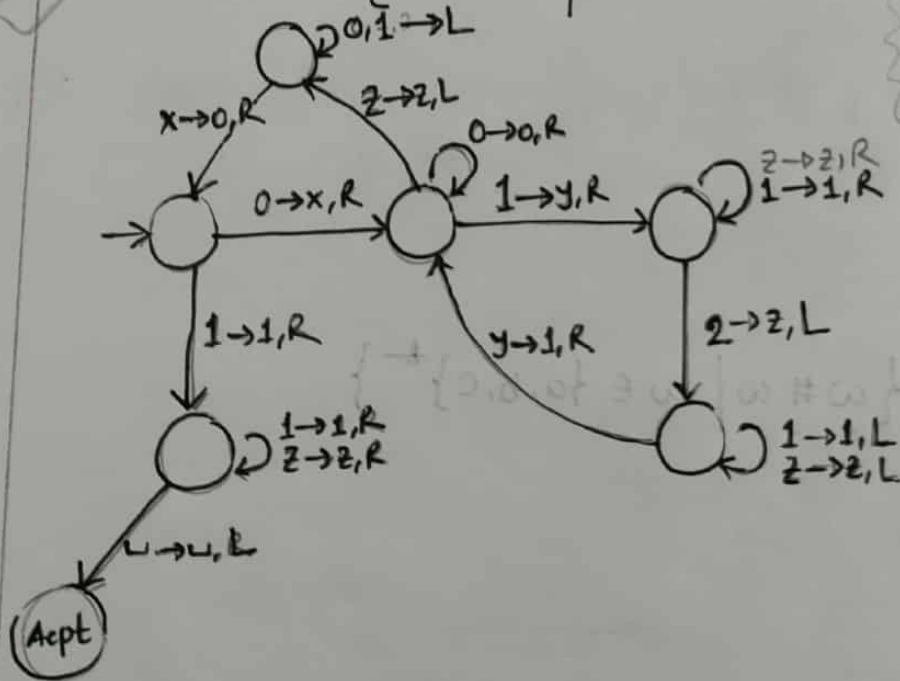
Example 13 - $L = \{w(a/b)w^R \mid \text{odd palindrome}\}$ $\Sigma = \{a, b\}$



tape:

a b a b a b a b a
u u u u u u u u

Example - 14: $\{0^i 1^j 2^k \mid i, j, k \geq 0 \text{ and } k = i + j\}$ $\Sigma = \{0, 1, 2\}$


$$i \otimes j = i + i + \dots + j + \dots + j$$
 $\theta^2 1^3 2^6$

00111222222

x0111 222222

x 0411 22 2222

xoyii z22222

x01y1 z72222

x011y zzzzz

✓ x011122222

$$282.12$$

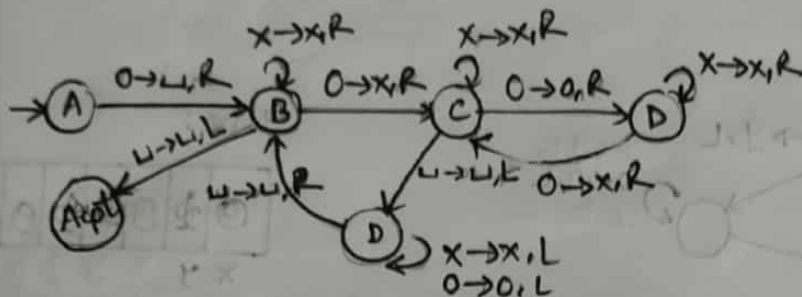
0x111 777777

0x yll zzzz

OXIII-22225

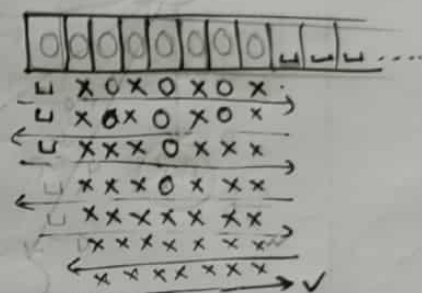
00111 2222

Example - 15 $L = \{0^{2^n} \mid n \geq 0\}$ where $\Sigma = \{0\}$

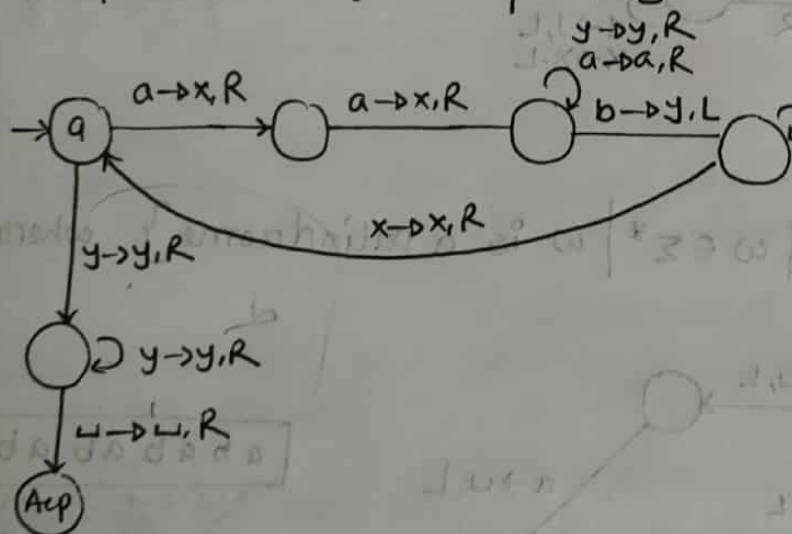


$n=0, 0$
 $n=1, 00$
 $n=2, 0000$
 $n=3, 00000000$

tape:

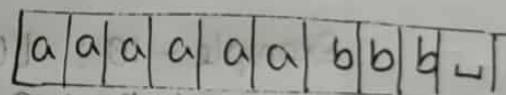


Example 16: $L = \{a^{2^n} b^n \mid n \geq 1\}$

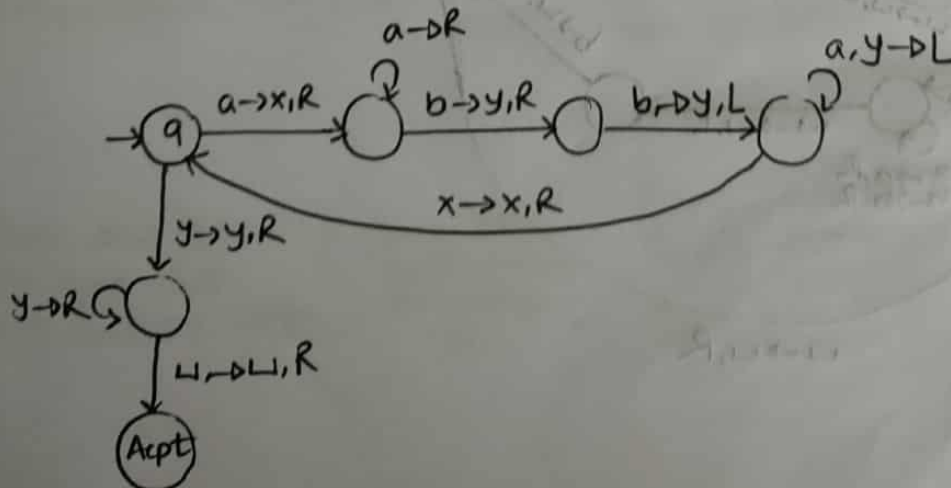


Ex: aaaabb

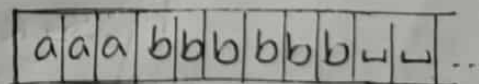
Tape:



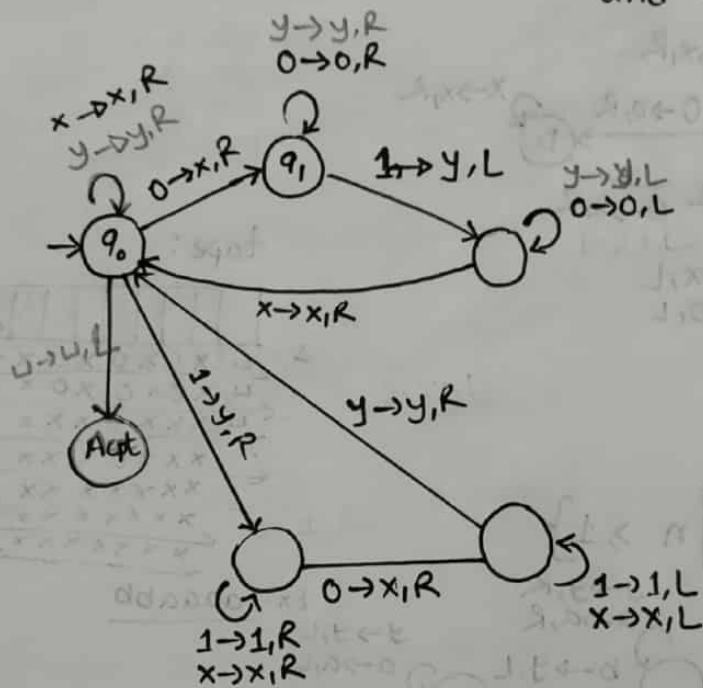
Example 17: $L = \{a^n b^{2^n} \mid n \geq 1\}$



tape:



Example - 18: $L(M) \rightarrow \{w \in \Sigma^* \mid w \text{ contain equal number of 0's and 1's}\}$ where $\Sigma = \{0, 1\}$



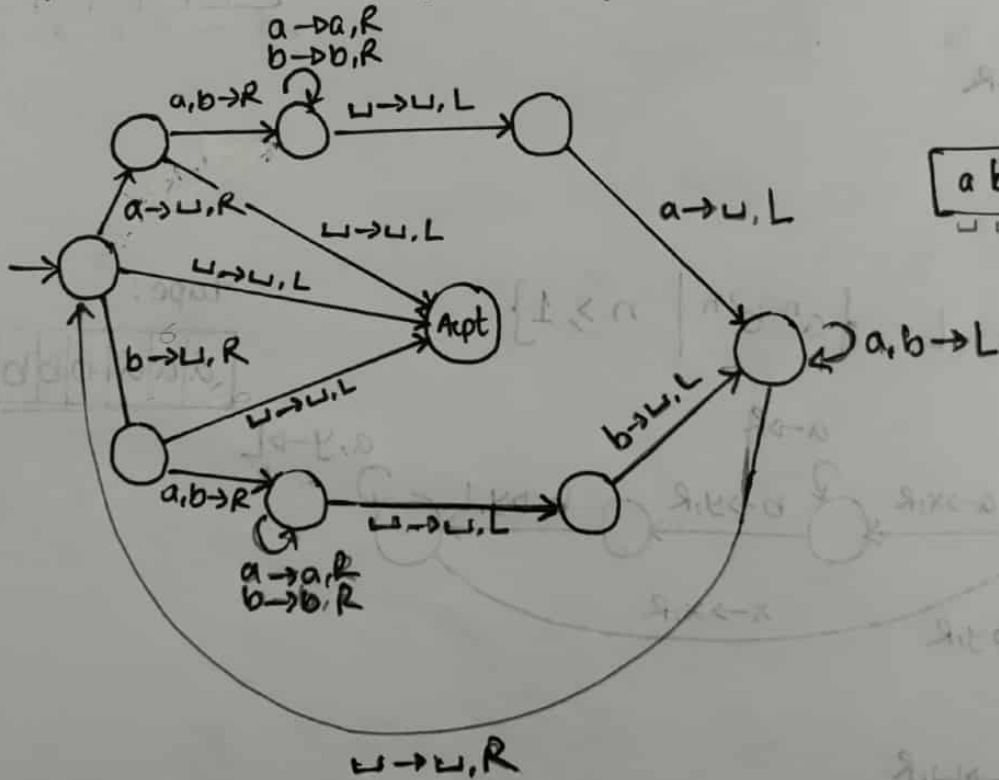
Lape :

0	1	0	1	1	0	1	0	_	_
x	y	x	y	y	x	y	x		

0 0 0 1 0 1 1 1 0 1
x x x y x y y y x y

1 1 0 1 0 0 0 1
y y x y ~~x~~ x x y

Example - 19: $L(M) \rightarrow \{w \in \Sigma^* \mid w \text{ is a palindrome}\}$ where $\Sigma = \{0,1\}$



ababababa

The Church Turing Thesis

↳ Anything that is computable (calculate by Algo) can be computed by TM.

↳ Any TM is $\equiv$ to computer, as it accept any Algo.

Decidability and undecidability

$\begin{cases} \text{Accept} \rightarrow \text{Halt} \\ \text{Reject} \rightarrow \text{Halt} \end{cases}$

Recursive lang $\rightarrow$

- Accept all str in 'L'
- Reject all str not in 'L'
- Halt every time and give an ans. for each input

$\begin{cases} \text{Accept} \rightarrow \text{Halt} \\ \text{Reject} \rightarrow \text{Halt may or may not} \end{cases}$

Recursively Enumerable $\rightarrow$

- Accept (and Halt) for all inpt.
- But may or may not halt for all input str not in L.

Decidable lang $\rightarrow$

- if recursive lang.
- All decidable $\Leftrightarrow$ Recursive lang.

(Acc/Rej after Halt)



Partially Decidable lang $\rightarrow$ • if recursive Enumerable lang.

Undecidable lang $\rightarrow$

- if not decidable
- may sometime partially decidable
- Not even partially decidable $\rightarrow$ Exist NO TM for that lang.

There are some TM that fall into forever loop. } There are TM that does not stop.

Accept $\rightarrow$ Halt
Reject $\rightarrow$ Halt
 $\rightarrow$ loop

3) A) $L = \{ \langle M, w \rangle \mid \text{The TM } M \text{ halts on input } w \text{ within } N \text{ steps} \}$

Here is a Turing Machine T that decides the language:

Turing Machine T on input $\langle M, w \rangle$:

1. First check the input $\langle M, w \rangle$ where M is a Turing machine and w is a string. If this is false Reject.
2. Simulate TM M on an input of w for N steps.
3. If this Turing machine M halts within N steps then Accept.
If not halt in N steps, then Reject.

As the simulation always ends in N steps, our Turing Machine T is a decider.

Hence, the language is decidable: if $w \in L(M)$, T accepts.
if $w \notin L(M)$, T rejects.

B) $L(M) = \{ \langle G \rangle \mid G \text{ is a connected undirected graph} \}$

Here is a TM T that decides the language:

Turing Machine T on input $\langle G \rangle$:

1. First check the input $\langle G \rangle$ where G is a finite graph. If not Reject.
 $\langle G \rangle$ encodes a finite Graph $G = (V, E)$.

2. Check for connection. Perform a graph traversal (DFS/BFS) starting from any vertex.
if every vertex is reached during the traversal, accept. Otherwise, Reject.

Since the graph is finite, the graph traversal stops after visiting all reachable vertices. Thus the machine always halts.

- If the graph is connected, T accepts.
- If the graph is disconnected, T rejects.

Therefore, the language is decidable.

4) a) The collection of decidable language is closed under union.

Let L_1 and L_2 be decidable by deciders T_1 and T_2 .
We want to show that $L_1 \cup L_2$ is also decidable.

Define a machine T :

$T =$ On input w :

1. Simulate T_1 on w .

- If T_1 accepts, then accepts.

• If T_1 rejects, run T_2 on w .

- If T_2 accepts, then accepts.

• If T_2 rejects, then rejects.

Both T_1 and T_2 halts on every input, so T also halts and gives the right result.

- If $w \in L_1 \cup L_2$, T accepts.
- If $w \notin L_1 \cup L_2$, T rejects.

Hence, $L_1 \cup L_2$ is decidable.

always.

b) The collection of decidable language is closed under complement.

Suppose L is a decidable language and T_1 is a Turing Machine that decides L .

To prove that decidable language are closed under complement, we can build another machine that decides $\bar{L}$.

Let the machine be T .

T = on input w :

1. Run T_1 on w .

- if T_1 accepts, then reject
- if T_1 rejects, then accept

Since T_1 is a decider, it halts on all input. Therefore T also halts and gives the correct answer.

- if $w \in L$, then T rejects.
- if $w \notin L$, then T accepts.

Thus, $\bar{L}$ is decidable.